

ITIL · DEVOPS FOR PRACTITIONERS

DevOps for ITIL

Structured Exam Study Guide & Scenario Practice

CAMS & Three Ways

Focus indicators

Diagrams & flows

Multi-answer practice

SPEED · AND · QUALITY

Prepared 2026 · Based on your DevOps for ITIL notes

How to Use This Guide

Your exam is **scenario-based** and **more than one option can be correct**. In DevOps-for-ITIL the marks live in three places: (1) knowing the **core models cold** — CAMS and the Three Ways; (2) matching a **named practice** to the problem it solves (Blue-Green, Chaos Monkey, Andon Cord, ICS, ChatOps); and (3) telling apart **look-alike pairs** — Utility vs Warranty, Pets vs Cattle, CAB vs Change Authority, SLA vs SLO, CapEx vs OpEx, RACI vs SOAR, Incident vs Problem. Treat every option as its own true/false statement.

READ THE TRIGGER
"break it on purpose" → Chaos Monkey · "switch back instantly" → Blue-Green · "stop the pipeline" → Andon Cord · "don't blame, find why" → Blameless Postmortem.

JUDGE EACH OPTION
Several statements can be true together — don't stop at the first correct one.

WATCH THE TRAPS
Each topic has an "Exam Trap" box showing the exact distinction examiners test.

Focus Indicator — what the signs mean

- HIGH FOCUS

Very likely in scenarios · master fully
- MEDIUM FOCUS

Common distinctions · know the differences
- SUPPORTING

Context & recall · lighter revision

Focus Priority Map

Topic	Focus	Why it matters for scenarios
CAMS (Culture · Automation · Measurement · Sharing)	●●● HIGH	The 4 DevOps values — the single most tested model.
The Three Ways (Systems · Feedback · Learning)	●●● HIGH	Gene Kim's principles — expect a full question.
DevOps Practices (Blue-Green, Chaos Monkey, Andon...)	●●● HIGH	Match the named practice to the scenario problem.
DevOps Methods (People→Process→Tools, CD, SRE, IaC)	●●● HIGH	SRE, IaC and tool-order questions are common.
DevOps Tool Chain (Programmable · Verifiable · Well-Behaved)	●●● HIGH	"UI alone is not enough" — the classic catch.
Pets vs Cattle · Variation & Dependencies	●●● HIGH	Standardise; the two "enemies" of fast delivery.
Service Lifecycle & Process mapping (Transition/Ops)	●●○ MED	Which DevOps practice improves which ITIL stage.
Utility vs Warranty · Wall of Confusion	●●○ MED	Dev=Utility=features; Ops=Warranty=stability.
SLA→SLO · CapEx→OpEx · Demand vs Capacity	●●○ MED	Cloud-era shifts in Design & Strategy processes.
Enterprise DevOps (Hybrid, Legacy, Scale, Silos)	●●○ MED	DevOps + ITIL + legacy in large organisations.
RACI vs SOAR · Roles · Just-in-Time access	●●○ MED	Shared ownership ≠ no ownership.
9 benefits · detailed step lists · terminology recall	●○○ SUP	Recognise, don't rote-memorise.

1 · Why DevOps for ITIL? — Speed AND Quality

●●○ MEDIUM

THE CORE PROBLEM

Dev wants **speed** (release changes fast); Ops wants **stability** (keep systems up). Traditionally you chose one.

THE DEVOPS MINDSET

Old: Speed **OR** Quality ❌ → **New:** Speed **AND** Quality
✅ . Deliver value faster *without* sacrificing stability.

★ ONE-LINE TAKEAWAY

DevOps + ITIL = deliver value **faster, with quality**, through **collaboration, automation, and continuous improvement**.

The 9 benefits of ITIL + DevOps

Benefit	Remember
Faster service delivery	CI/CD + automation → services reach customers faster
Better collaboration	Removes silos → one team, one goal
More agile	Respond to changing business needs quickly
Automation	Less manual work = fewer mistakes
Continuous improvement	Short iterations (sprints) → quicker improvement
Better monitoring	Find problems before customers do
Better culture	Shared responsibility, focus on outcomes
Better customer value	Faster + high quality = happy customers
Better career skills	Dev + Ops + Service Management = more valuable

MEMORY TRICK

DevOps doesn't change ITIL's **goal** (deliver value) — it upgrades the **how** to be faster and automated.

2 · Traditional IT vs DevOps

●●○ MEDIUM

UTILITY VS WARRANTY — MEMORISE THIS PAIR

Dev = Utility = Features ("What the service does"). · **Ops = Warranty = Reliability** ("How well it works"). DevOps achieves both together.

★ EXAM TERM — WALL OF CONFUSION

The **separation between Dev and Ops** that causes miscommunication, delays, blame, and slow delivery. Dev throws code "over the wall" to Ops. DevOps tears down this wall.

Why they were separated — and what changed

THEN (PHYSICAL ERA)

Releases were large, slow, manual, often physical (DVDs/CDs). A mistake shipping 1M DVDs was **costly**, so separation & many checks made sense.

NOW (CLOUD / DIGITALISATION)

Digital releases, small updates, fast releases, easy rollback, high automation → changes many times a day. **Cloud enables DevOps.**

Traditional IT	DevOps
Separate Dev & Ops	Shared responsibility (one team)
Large releases · slow · manual	Small frequent releases · fast · automated
Physical software	Digital services
Utility vs Warranty handled separately	Utility & Warranty achieved together

MEMORY TRICK

Traditional: Separate → Slow → Stable. **DevOps:** Collaborate → Automate → Deliver faster. Automation is best for work that is **repetitive, frequent, low-risk**.

3 · DevOps vs Enterprise DevOps

●●○ MEDIUM

THE EQUATION TO REMEMBER

Enterprise DevOps = DevOps + Enterprise IT + ITIL. It applies DevOps in **large organisations** that still have legacy systems, ITIL processes, and **hybrid (cloud + traditional)** environments.

DevOps (Cloud-native)	Enterprise DevOps
Cloud-first	Hybrid (cloud + legacy)
Highly automated	Partially automated
Few restrictions	ITIL processes & approval gates
Modern systems · developers deploy	Legacy systems exist · Ops controls deployments
One team per service	Separate teams to integrate

ROLE OF ITIL INSIDE ENTERPRISE DEVOPS

Provides **service management, change management, governance, risk control**. DevOps supplies the speed; ITIL supplies the control.

Resources · Capabilities · Constraints (know the difference)

RESOURCES

What you **have**: people, money, tools, technology.

CAPABILITIES

How well you **use** resources: skills, processes, automation, teamwork.
Higher capability → better results.

CONSTRAINTS

What **limits** you: budget, time, legacy systems.

MEMORY TRICK

Resources = what you have · **Capabilities** = how well you use it · **Constraints** = what's stopping you.

4 · The Two Enemies & Pets vs Cattle ★

... HIGH

COMMON GOAL OF DEVOPS & ENTERPRISE DEVOPS

Fast Time to Value + Quality + Continuous Improvement. Both reduce variation and dependencies to get there.

The two biggest enemies of fast delivery

A · UNNECESSARY VARIATION

Things differ when they needn't (different configs, versions, settings). → more discussion, more errors, harder troubleshooting. **Solution: standardise.** Less variation = less confusion.

B · UNNECESSARY DEPENDENCIES

One team/system waits on another (Team A can't release until Team B finishes). → delays, large releases, more meetings. **Solution: make teams/services independent.** Less dependency = faster delivery.

★ Pets vs Cattle — a favourite exam term

PETS ❌

Every server is **unique** — own name, own settings, manual care. Hard to automate.

CATTLE ✅

All servers are **identical** — standardised, easily replaceable, easy automation. **DevOps prefers Cattle.**

MEMORY TRICKS

Variation → Confusion · Dependency → Delay · Pets = Unique ❌ / Cattle = Standardised ✅. Reducing both variation and dependencies → smaller, faster, higher-quality releases.

5 · DevOps Values — CAMS ★★☆☆ (Master)

●●● HIGH — TOP TOPIC

THE 4 CORE DEVOPS VALUES

DevOps is built on **CAMS**: **C**ulture · **A**utomation · **M**easurement · **S**haring. Expect a question that asks you to identify or apply one of these.

Letter	Value	Purpose	ITIL connection
C	Culture	Collaboration & no blame	Blameless postmortems after incidents/failed changes
A	Automation	Reduce manual work & errors	Automate testing, deployment, service requests, infra
M	Measurement	Use data to improve	Continual Improvement · MTTR, CSAT, cost, performance
S	Sharing	Share knowledge & tools	Knowledge Management

CULTURE — BLAMELESS

When something breaks: don't blame people → find the **root cause** and improve. **Fix the process, not the person.**

AUTOMATION

Benefits: faster, fewer errors, less variation, self-service. Automate **people's repetitive work, not people.**

MEASUREMENT

Use data, not opinions. "**If you can't measure it, you can't improve it.**"

SHARING

Share knowledge, tools, code, lessons → less duplicate work, faster learning.

★ EXAM TRAPS

Automation ≠ automating people out. It targets repetitive tasks. And **Measurement is data-driven** — an option saying decisions are based on opinion/seniority is wrong. Blameless culture belongs to **C**, not **A**.

6 · DevOps Principles — The Three Ways ★★

... HIGH — TOP TOPIC

THE THREE WAYS (BY GENE KIM)

1 Systems Thinking → 2 Amplify Feedback Loops → 3 Continuous Experimentation & Learning. Memory: **System** → **Feedback** → **Learning**.

1 · Systems Thinking — optimise the **WHOLE** value stream



2 · Amplify Feedback Loops — fast feedback, then improve



3 · Continuous Experimentation & Learning — learn from small failures

1 · SYSTEMS THINKING

Improve the **whole**, not one team.
Don't pass problems downstream.
Avoid local optimisation. Applies across the entire Service Lifecycle.
Think **end-to-end**.

2 · FEEDBACK LOOPS

Get feedback **quickly** (customers, testing, monitoring, ops) → faster fixes, better quality. **Shorter feedback = faster improvement.**

3 · EXPERIMENT & LEARN

Keep experimenting; accept that **small failures aid learning**; practice builds expertise → innovation & continual improvement.

★ EXAM TRAP

The First Way is about the **whole system / end-to-end flow** — an option praising optimising a single team or department in isolation is *against* Systems Thinking (that's the local-optimisation trap).

MEMORY TRICK

1 **System** · 2 **Feedback** · 3 **Learning**. Experiment → Learn → Improve.

7 · DevOps Methods

... HIGH — TOP TOPIC

★ #1 METHOD TRAP — PEOPLE → PROCESS → TOOLS

Correct order: decide **who** (People) → define **how** (Process) → pick the **tool last**. Buying a tool first is the **wrong** answer. **People first**.

Method	Meaning	ITIL benefit
People → Process → Tools	Choose tools last	Better process design
Continuous Delivery (CD)	Smaller + faster = better : small, frequent, safe releases	Faster Service Design & Transition
Lean	Remove waste , improve continuously	Efficient services
Agile	Respond to change fast, short cycles	Faster improvement
Visible Ops Change Control	Lightweight change mgmt: reduce fragile systems & manual work, standardise builds	Modernises Change Enablement
Infrastructure as Code (IaC)	Treat infrastructure like software code — write code, deploy automatically	IT Service Continuity & Disaster Recovery
Site Reliability Engineering (SRE)	Software Engineering + Operations : automate ops, improve reliability, RCA, watch SLOs	Incident, Problem, Availability, CSI

LEAN VS AGILE

Lean = Remove Waste · Agile = Adapt Fast.

SRE & IAC

SRE = Developer + Operations = Reliability. **IaC: Infrastructure = Code** → consistent environments, easy DR.

★ SRE WATCHES SLOS

SRE performs Root Cause Analysis and monitors **Service Level Objectives (SLOs)**. Link SRE to reliability every time it appears.

8 · DevOps Practices ★★☆☆ (match to scenario)

●●● HIGH — TOP TOPIC

HOW THIS IS TESTED

A scenario describes a **problem or goal**; you pick the practice(s) that fit. Learn the trigger phrase for each.

Practice	Purpose / trigger phrase
Incident Command System (ICS)	Structured handling of major incidents — clear roles, scales up/down
Devs on Call	" You build it, you support it " → better quality, fast feedback
Public Status Pages	Show service health/outages → transparency & customer trust
Blameless Postmortems	After incident: find root cause, don't blame → learn & prevent
Embedded Teams	Dev + Ops (+QA/Security) in one team → less handoff
Cloud Adoption	Enables automation, IaC, faster deployments
Andon Cord ★	Emergency stop button — anyone can stop the build/release on a serious defect
Dependency Injection	Reduce dependencies between components → easier, safer changes
Blue-Green Deployment ★	Two environments; switch to Green, switch back to Blue if it fails → zero-downtime, fast rollback
Chaos Monkey ★★☆☆	Netflix tool — intentionally break systems to test resilience & find weaknesses first

BLUE-GREEN

Blue = Old · Green = New · switch back if needed. Safe deployment + fast rollback + minimal downtime.

CHAOS MONKEY VS ANDON CORD

Chaos Monkey = break it on purpose to build resilience.
Andon Cord = see a problem → stop the pipeline.

★ EXAM TRAP

Don't confuse them: **Blue-Green** = safe release/rollback; **Chaos Monkey** = resilience testing by causing failures; **Andon Cord** = human emergency stop; **ICS** = major-incident coordination.

9 · DevOps Tools & the Tool Chain ★★

... HIGH

KEY IDEA

DevOps is **not one tool** — it is a **Tool Chain**: connected tools that automate the lifecycle (SCM, build, CI, testing, deployment, monitoring, security, cloud). **Tool Chain = Connected Tools**.

★ Three characteristics of a good tool chain — P · V · W

PROGRAMMABLE

Supports **UI + CLI + API** so it can be automated.

VERIFIABLE

Shows what it did & lets you validate results → trust the automation.

WELL-BEHAVED

Integrates easily with other tools; supports Dev & Ops → smooth collaboration.

★ EXAM TRAP

A tool with **UI only** is **NOT enough** — "Programmable" specifically requires **API + CLI** as well, so it can be scripted into the pipeline.

Configuration Management — ITIL vs DevOps

Traditional ITIL	DevOps
Record IT components in a CMDB	Store infrastructure/config in Version Control (Git) — treat config like code

MEMORY TRICKS

Good tool chain = **P·V·W**. Config Mgmt: **ITIL** → **CMDB** · **DevOps** → **Git/Version Control**. Good DevOps tool = **UI + API + CLI**. Servers: **Pets** → **Cattle**.

10 · Enterprise Challenges & How DevOps Answers Them

●●○ MEDIUM

Change control & release gates

THE KEY IDEA

More stable/consistent environment → **less approval needed**. More variation → more approvals. So reduce variation with **automation (A in CAMS)** and **systems thinking** (whole pipeline, "trust but verify").

Automation + Standardisation → CI/CD → Blue-Green Deployment → Dependency Injection → ChatOps

CHATOPS ★

Collaboration through **chat + bots + automation** (e.g. Slack/Teams bots deploy code, run tests, create tickets). Brings **tools into team conversations**.

LEGACY / REGULATED / NON-AUTOMATED

Move to **cloud when it makes business sense**; automate only where valuable; **standardise** what can't move; use **common processes** (same change process, different tech); share tools; use ChatOps. Good tool = **UI+API+CLI**.

SILOED (DISPARATE) TEAMS

Break barriers with shared goals & **Culture (C)**. Use **Value Stream Mapping** (idea→customer, find bottlenecks), reduce dependencies (containers, versioned APIs, data contracts), **Embedded Teams**, **Devs on Call**, **Blameless Postmortems**.

MASSIVE SCALE

Start Small → **Improve** → **Repeat** → **Scale**. Share a showcase project (S in CAMS), use Systems Thinking (avoid local optimisation), Value Stream Mapping, **ICS** for major incidents, ChatOps, and improve the build pipeline continuously.

MEMORY TRICK

Value Stream Mapping = map idea→customer to find delays & bottlenecks. Appears in *silos* and *scale*.

11 · ITIL Service Lifecycle + DevOps ★★

●●○ MEDIUM

5 STAGES — S · D · T · O · CI

Strategy → **Design** → **Transition** → **Operation** → **Continual Improvement**. DevOps improves **every** stage (faster, automated, continuously improving).

Stage	Purpose (Plan/Build/Deploy/Run/Improve)	DevOps impact
Strategy	Decide what services to offer (Plan)	Faster feedback & experimentation
Design	Design the service (Build)	Cloud, Agile, CI/CD, microservices
Transition	Build & deploy (Deploy)	Automation, Blue-Green , CI/CD, Visible Ops
Operation	Run the service (Run)	SRE , Devs on Call, ICS, monitoring
Continual Improvement	Keep improving (Improve)	Measurement & feedback, Lean/Agile

Terminology — three levels of change

NO CHANGE

Some terms stay identical (e.g. Patterns of Business Activity).

HOW CHANGES

Same goal, faster method: large slow releases → small frequent releases.
Value still delivered, faster.

WHAT CHANGES

Concept changes completely:
manage servers (**Pets**) → manage code/automation (**Cattle**).

DEVOPS REPLACES SOME TRADITIONAL PRACTICES

Major Incident Process → **ICS** · Traditional Release → **Blue-Green** · Heavy ticketing → **ChatOps**. **ITIL = What & Why** · **DevOps = better How**.

Strategy processes

ITIL process	DevOps shift
Strategy Management	Faster planning & feedback (sprints, retrospectives)
Service Portfolio Management	Services → Features ; measure if customers actually use features
Demand Management	Cloud = Demand, not Capacity ; remove unused resources
Financial Management	CapEx → OpEx (pay-per-use); usage tracking & forecasting
Business Relationship Mgmt (BRM)	Continuous customer feedback → better features

Design processes

ITIL process	DevOps shift
Design Coordination	Design with Operations in mind (SRE), CI/CD, self-service
Service Level Management	SLA → SLO ; user experience, analytics, status pages
Availability Management	Automation, SRE, ICS, Visible Ops
Capacity Management	Cloud = auto/elastic scaling ; demand forecasting
Supplier Management	Manage subscriptions , avoid paying for unused
Information Security Mgmt	Identity, app & data security, automated security (less network-only)
Service Catalog Management	Catalog → Self-Service Portal
IT Service Continuity (ITSCM) ★	IaC + Blue-Green → fast DR, quick rebuild, easy failover

MEMORY TRICKS

SLA→SLO · CapEx→OpEx · Capacity→Demand · Catalog→Self-Service · ITSCM = Everything as Code = faster recovery.

Transition processes

ITIL process	DevOps shift
Change Management ★	Smaller frequent changes, Visible Ops , automation — enable change, don't block it
Release & Deployment ★	Small + Fast + Automated ; CD, Blue-Green; Product Owner approves release
Knowledge Management	Documents → ChatOps , real-time sharing
Service Validation & Testing	Continuous automated testing
SACM ★★ ★	CMDB → Code (scripts, templates, pipelines, IaC); Pets → Cattle ; continuous discovery

Operations processes

ITIL process	DevOps shift
Incident Management ★★ ★	ICS + automation → lower MTTR ; treat incidents as bugs to fix
Problem Management ★★	SRE , ChatOps, data-driven RCA → permanent fixes
Event Management ★★	Cloud monitoring, telemetry, analytics, status pages
Request Fulfillment ★	Manual → Self-Service
Access Management ★	Standing access → Just-in-Time (On-Demand) access → better security

★ INCIDENT VS PROBLEM

Incident = restore service now. Problem = find the root cause (why). Don't merge them in a scenario.

MEMORY TRICK

SACM: CMDB → **Code**, **Pets** → **Cattle**. **Access: On-Demand, not standing.** **MTTR** = Mean Time To Repair (lower is better).

14 · CSI, Functions & Roles + DevOps

●●○ MEDIUM

CONTINUAL SERVICE IMPROVEMENT (CSI)

Measure → **Learn** → **Improve** → **Repeat**. DevOps adds faster feedback loops, Lean & Agile, SRE, measurement, automation → **data-driven decisions**.

THE 4 ITIL FUNCTIONS — S T A O

Service Desk (SPOC → self-service, chatbots) · **T**echnical Mgmt (cloud, IaC, SRE, Chaos Monkey) · **A**pplication Mgmt (CD, automation, monitoring) · **O**perations (ChatOps, automation, cloud). All become more automated, collaborative, cloud-based.

Roles — shared ownership ≠ no ownership

Role	Responsibility
Process Owner	Accountable — owns & improves the process
Process Manager	Runs the process day-to-day (DevOps: trust, support, verify — not block/delay)
Process Practitioner	Does the actual work (more automated, still needed)
Service Owner	Owens a service end-to-end (health, quality, improvement)

★ RACI vs SOAR

RACI (ITIL)

Responsible (does) · **A**ccountable (owns) · **C**onsulted (advises) · **I**nformed (updated).

SOAR (DEVOPS ALT.)

Shared **O**wnership and **R**oles — more collaboration, less rigid separation. ITIL uses RACI; DevOps may prefer SOAR.

MEMORY TRICK

Everyone helps. Someone owns. Exactly one **A** (Accountable) per activity in RACI.

15 · Technology & Architecture + Master Revision

●●○ MEDIUM

TECHNOLOGY & ARCHITECTURE

DevOps doesn't replace it — it **adds automation everywhere** (CI/CD, cloud, IaC, containers, monitoring, logging, security). **Shift automation left** (earlier into development). Event Mgmt techniques stay useful: compression, correlation, filtering, intelligent monitoring, roll-up, verification — and use the **same monitoring in pre-production & production**.

REDUCE VARIATION ★

ITIL & DevOps agree: **more variation = lower quality**. Automation → same deployment, same config, same tests every time. **Automation = Consistency**.

One-glance look-alike pairs (highest yield)

Pair	The distinction
Utility vs Warranty	Features (what) · vs · Reliability (how well)
Pets vs Cattle	Unique manual servers · vs · Standardised replaceable
Blue-Green vs Chaos Monkey	Safe release/rollback · vs · Break on purpose (resilience)
Andon Cord vs ICS	Stop the pipeline · vs · Coordinate a major incident
Incident vs Problem	Restore now · vs · Find root cause
SLA vs SLO	Agreement (contract) · vs · Objective (target SRE watches)
CapEx vs OpEx	Upfront buy · vs · Pay-per-use (cloud)
Capacity vs Demand	Add capacity manually · vs · Cloud auto-scales to demand
RACI vs SOAR	ITIL roles model · vs · DevOps shared-ownership model
Variation vs Dependency	Confusion/errors · vs · Delay/waiting

GOLDEN THREADS

CAMS · Three Ways · Speed AND Quality · Reduce variation & dependencies · Automate to reduce approvals · Blameless culture.

Scenario Practice — Set A (more than one statement may be correct)

Q1 A retailer keeps missing releases because Dev pushes code and Ops refuses to deploy it near peak trading, blaming each other when things break. Which statements correctly diagnose and address this?

- A. This is the "Wall of Confusion" — separation between Dev and Ops causing delays and blame.
- B. Embedded Teams and shared goals (Culture in CAMS) would reduce the handoff.
- C. The fix is to make Ops solely responsible for all future failures.
- D. Blameless Postmortems would shift focus from "who" to "why".

Correct: A, B, D. C is wrong — DevOps removes blame and shares responsibility, it doesn't concentrate blame on one team.

Q2 A bank wants to deploy a new payments release with the ability to revert instantly if it fails, and separately wants confidence the platform survives component failures. Which practices fit?

- A. Blue-Green Deployment for instant rollback to the old version.
- B. Chaos Monkey to intentionally break components and test resilience.
- C. Andon Cord so anyone can stop a release when a serious defect is seen.
- D. Pets approach — hand-configure each server for the release.

Correct: A, B, C. D is wrong — DevOps prefers Cattle (standardised, automated), not Pets.

Q3 During tool selection a manager says "buy this platform first, then we'll figure out the process and who runs it." Which statements are correct?

- A. This reverses the DevOps order; it should be People → Process → Tools.
- B. A good tool must be Programmable — UI alone is not enough; it needs API + CLI.
- C. Tools should also be Verifiable and Well-Behaved.
- D. Choosing the tool first is the recommended DevOps method.

Correct: A, B, C. D is wrong — tools come *last*.

Q4 A team optimises its own stage to be very fast, but overall delivery to the customer gets slower because the next team is now overwhelmed. Which apply?

- A. This violates Systems Thinking (First Way) — optimise the whole, not one part.
- B. It is an example of local optimisation, which DevOps warns against.
- C. Value Stream Mapping would reveal the downstream bottleneck.
- D. Speeding up one team always improves end-to-end flow.

Correct: A, B, C. D is wrong — local speed-ups can create bottlenecks elsewhere.

Scenario Practice — Set B (more than one statement may be correct)

Q5 A large enterprise runs both cloud services and old mainframe systems, with ITIL change approvals. How should you describe and approach this?

- A. This is Enterprise DevOps = DevOps + Enterprise IT + ITIL.
- B. It is a hybrid environment (cloud + legacy).
- C. Automate everything immediately, including every legacy system.
- D. Move to cloud when it makes business sense; standardise what can't move.

Correct: A, B, D. C is wrong — automate only where it adds value; don't force automation onto every legacy system.

Q6 Leadership asks how DevOps reduces the heavy change-approval burden without increasing risk. Which statements are correct?

- A. A more stable, consistent environment needs less approval.
- B. Reducing unnecessary variation (via automation/standardisation) reduces change control.
- C. CI/CD and Blue-Green make releases safer and faster, easing gates.
- D. The only way to speed approvals is to remove all governance.

Correct: A, B, C. D is wrong — governance stays; DevOps reduces the *effort* via consistency, not by removing control.

Q7 An outage recurs weekly. The team keeps restarting the service to restore it but the failure returns. Which statements are correct?

- A. Restarting to restore service is Incident Management.
- B. Finding the underlying root cause is Problem Management.
- C. SRE with data-driven RCA and ChatOps supports the permanent fix.
- D. Incident and Problem Management are the same activity here.

Correct: A, B, C. D is wrong — Incident = restore now; Problem = find why. They are distinct.

Q8 A CIO wants DevOps decisions and improvements to be based on facts, and wants knowledge shared across teams. Which CAMS values and effects apply?

- A. Measurement — use data (MTTR, CSAT, performance), not opinions.
- B. Sharing — share knowledge, tools, code, lessons learned.
- C. Measurement supports Continual Improvement; Sharing supports Knowledge Management.
- D. Automation means replacing the people rather than their repetitive tasks.

Correct: A, B, C. D is wrong — Automation targets repetitive *work*, not the people.

Scenario Practice — Set C (more than one statement may be correct)

Q9 A company moves from buying data-centre hardware upfront to a cloud model, and wants resources to match real usage. Which statements are correct?

- A. The financial shift is CapEx → OpEx (pay-per-use).
- B. Focus moves from Capacity Management to Demand Management as cloud auto-scales.
- C. Unused resources should be removed and usage monitored.
- D. Cloud means capacity must always be added manually in advance.

Correct: A, B, C. D is wrong — cloud scales elastically to demand; manual pre-provisioning is the traditional model.

Q10 A team treats servers as unique, hand-tuned machines with different configs, causing errors and slow troubleshooting. Which statements apply?

- A. These are "Pets"; DevOps prefers "Cattle" — standardised, replaceable servers.
- B. Storing configuration in version control (config as code) reduces variation.
- C. In DevOps, SACM shifts from CMDB records toward scripts, templates and IaC.
- D. Unnecessary variation improves service quality.

Correct: A, B, C. D is wrong — more variation lowers quality; automation brings consistency.

Q11 To improve security, admins should only get elevated access when needed and only for as long as needed. Which statements are correct?

- A. This is Just-in-Time (On-Demand) access, a DevOps shift in Access Management.
- B. It replaces permanent "standing" access and lowers risk.
- C. Request Fulfillment can move from manual handling to self-service.
- D. Standing permanent access for everyone is the more secure option.

Correct: A, B, C. D is wrong — standing access increases risk; on-demand access is more secure.

Q12 A disaster wipes an environment; the business needs to rebuild it quickly and reliably. Which DevOps methods/practices help most?

- A. Infrastructure as Code (IaC) to rebuild environments automatically and consistently.
- B. Blue-Green Deployment to fail over with minimal downtime.
- C. These support IT Service Continuity Management (ITSCM).
- D. Manually rebuilding each unique server (Pets) is the fastest, most reliable route.

Correct: A, B, C. D is wrong — manual per-server rebuilds are slow and error-prone; "everything as code" enables fast recovery.

FINAL REVISION — SAY THESE OUT LOUD

CAMS · Three Ways (System→Feedback→Learning) · People→Process→Tools · Blue-Green = rollback · Chaos Monkey = break on purpose · Andon Cord = stop the line · Pets→Cattle · SLA→SLO · CapEx→OpEx · Incident=restore / Problem=root cause · Speed AND Quality.